

1. Introduction

This project implements a GraphRAG-based question answering system for a shared-housing domain. The goal of the system is to allow users to ask natural-language questions about apartments, roommates, owners, amenities, rules, rent prices, and locations, and receive answers that are grounded in a Neo4j knowledge graph.

The system combines two main components. First, it uses a Neo4j graph database to store and retrieve structured housing information. Second, it uses an LLM to help understand user questions and generate a final natural-language answer based only on the retrieved graph context.

The final pipeline follows the GraphRAG idea:

User Question → Question Understanding → Neo4j Graph Retrieval → Graph Context Serialization → Grounded LLM Answer

This approach makes the system more reliable than a simple LLM-only solution, because the final answer is based on explicit facts retrieved from the knowledge graph.

2. System Setup

The notebook begins by loading the required Python libraries and environment variables. The system connects to a Neo4j database using credentials stored in a `.env` file. It also optionally connects to the OpenAI API if an API key is available.

The main environment variables used are:

- `NEO4J_URI`
- `NEO4J_USER`
- `NEO4J_PASSWORD`
- `NEO4J_DATABASE`
- `OPENAI_API_KEY`
- `OPENAI_MODEL`

If the OpenAI API key is not available, the system can still run using deterministic logic for question understanding and answer generation. This makes the implementation more reproducible and easier to evaluate.

3. Question Normalization and Entity Extraction

Before querying the graph, the user question is normalized. This step improves the robustness of the system by handling small variations in the way users write questions.

For example, apartment identifiers such as `apt003`, `ap-003`, or `apt-003` are normalized into the standard form `apt-003`. The system also corrects some simple spelling mistakes, such as `amenties` to `amenities`.

The system extracts important entities and values from the question, such as:

- apartment IDs, for example `apt-001`
- owner IDs, for example `owner-001`
- rent amounts, for example `400`
- locations, such as `Ilioupoli`, `Glyfada`, or `City Center`
- amenities, such as `Wi-Fi`, `Parking`, or `Heating`
- roommate names, such as `Giorgos`, `Maria`, or `Kwstas`

This preprocessing step is important because it allows the system to map natural-language questions to structured graph queries more accurately.

4. Intent Detection

After normalization, the system tries to understand the intent of the user question. An intent represents the type of information the user is asking for.

Examples of supported intents include:

- finding apartments with rent below a given amount
- finding the rent of a specific apartment
- finding apartments in a specific location
- finding apartments with a certain number of bathrooms
- finding amenities of an apartment
- checking whether an apartment has a specific amenity
- finding roommates living in an apartment
- finding the owner of an apartment
- finding apartment rules
- checking whether pets are allowed
- answering combined questions about rules and amenities

The system first uses deterministic rules based on keywords, regular expressions, and extracted entities. If the deterministic logic cannot classify the question, the system can use an LLM to classify the question and extract parameters.

This hybrid design is useful because it combines the flexibility of an LLM with the stability of deterministic rules. For grading and evaluation, deterministic parsing is especially important because it produces consistent results.

5. Cypher Query Library

Each supported intent is connected to a predefined Cypher query. These Cypher queries retrieve the relevant information from the Neo4j knowledge graph.

For example, if the user asks:

“Which apartments have rent below 400 euros?”

the system maps the question to the intent `apartments_rent_below` and runs a Cypher query that finds all apartments where the rent is lower than the given amount.

The query library covers many different types of questions, including questions about:

- rent
- location
- bathrooms
- amenities
- Wi-Fi and parking
- roommates
- owners
- apartment rules
- pet allowance
- combined rules and amenities

Using predefined parameterized Cypher queries has two main advantages. First, it makes the system safer because the LLM does not generate arbitrary database queries. Second, it makes the system more reproducible because each intent always maps to a known query pattern.

6. Graph Retrieval and Context Construction

After the intent and parameters are identified, the system runs the corresponding Cypher query against the Neo4j database. The query returns structured records from the graph.

The system then extracts deterministic answer values from the query results. These values are used for evaluation because they are easier to compare against the expected answers than free-form LLM text.

In addition to the direct query results, the system also retrieves supporting graph triples related to the detected entities. These triples provide extra context about the relevant nodes and relationships in the graph.

The retrieved information is then serialized into a structured JSON context containing:

- the detected intent
- the extracted parameters
- the query records
- the supporting graph triples

This serialized graph context is the information passed to the LLM for final answer generation.

7. Grounded Answer Generation

The final answer is generated using only the retrieved graph context. The LLM is instructed not to use external knowledge and to answer only from the information provided by the graph.

This is the key GraphRAG part of the system. The LLM does not answer directly from its own general knowledge. Instead, it receives structured graph facts and produces a concise natural-language answer grounded in those facts.

For example, in the smoke test, the system receives the question:

“Which amenities does apartment apt-001 have?”

The graph returns the answer values:

Parking, Wi-Fi

The final generated answer is:

“Apartment apt-001 has the following amenities: Parking and Wi-Fi.”

This shows that the system successfully retrieves graph information and converts it into a clear natural-language response.

8. Unified QA Pipeline

The main function of the system is `run_question`.

This function takes a user question and returns three outputs:

1. `answer_values`: the deterministic answer values retrieved from Neo4j
2. `graph_context`: the retrieved graph records and supporting triples
3. `final_answer_text`: the final natural-language answer

The function can run in two modes.

When `use_rag=True`, the system follows the full GraphRAG pipeline and uses the LLM to generate a final grounded answer.

When `use_rag=False`, the system returns deterministic answer values only. This mode is useful for evaluation because it avoids variability from LLM-generated text.

9. Evaluation Methodology

The system is evaluated on two datasets:

- a clean evaluation dataset
- an adversarial evaluation dataset

The clean dataset contains normal questions written in a straightforward way. The adversarial dataset contains harder questions, including alternative wording, small typos, malformed apartment IDs, and combined queries.

For evaluation, the system compares the deterministic `answer_values` with the expected answers. Both predicted and expected answers are normalized before comparison, so small formatting differences do not affect the result.

This evaluation method is appropriate because the main goal is to check whether the system retrieves the correct facts from the graph.

10. Results

The final system achieved the following results:

- Clean Accuracy: **100%**
- Adversarial Accuracy: **100%**
- Robustness Score: **100%**

These results show that the system correctly answers all questions in both the clean and adversarial datasets.

The adversarial accuracy is especially important because it shows that the system can handle variations in user input. For example, it can deal with different apartment ID formats, simple spelling mistakes, paraphrased questions, and combined information requests.

The smoke test also confirms that the GraphRAG pipeline works end-to-end. The system retrieves relevant Neo4j facts and uses them to generate a grounded final answer.

11. Strengths of the System

One major strength of the system is that it combines deterministic logic with LLM support. The deterministic fallback makes the system stable and reproducible, while the LLM allows more flexible question understanding and answer generation.

Another strength is the use of a structured Neo4j knowledge graph. Since the answer comes from graph queries, the system is less likely to hallucinate compared to an LLM-only approach.

The predefined Cypher query library is also useful because it makes the retrieval process controlled and transparent. Each question type is mapped to a clear graph query, making the system easier to debug and evaluate.

Finally, the adversarial evaluation shows that the system is robust to realistic user input variations.

12. Limitations

Although the system performs very well on the provided datasets, there are some limitations.

First, the knowledge graph is relatively small and domain-specific. The system is designed for a shared-housing scenario and would need additional entities, relationships, and data to support a larger real-world application.

Second, the Cypher queries are template-based. This makes the system reliable, but it also limits flexibility. If the user asks a question outside the predefined intents, the system may not be able to answer it.

Third, the retrieval process is mostly symbolic and rule-based. The current system does not use vector search, graph embeddings, or semantic similarity search. These techniques could improve retrieval for more complex or ambiguous questions.

Finally, the evaluation datasets are useful but limited. A larger and more diverse benchmark would give a more realistic estimate of the system's performance.

13. Possible Future Improvements

Several improvements could make the system stronger.

First, the knowledge graph could be expanded with more apartments, roommates, owners, contracts, neighborhoods, prices, and user preferences.

Second, the system could support automatic Cypher generation. This would allow it to answer a wider range of questions, although it would also require stronger validation to avoid incorrect or unsafe queries.

Third, vector retrieval could be added. For example, apartment descriptions, roommate preferences, and house rules could be embedded and retrieved using semantic similarity.

Fourth, graph embeddings could be used to improve reasoning over relationships between apartments, roommates, owners, amenities, and rules.

Finally, the evaluation could be extended with more difficult multi-hop questions, such as questions that require combining rent, location, amenities, and roommate preferences at the same time.

14. Conclusion

This project successfully implements a GraphRAG question answering system for a shared-housing knowledge graph.

The system understands user questions, maps them to predefined intents, retrieves relevant facts from Neo4j, serializes the retrieved graph context, and uses an LLM to generate grounded natural-language answers.

The final results show perfect performance on both the clean and adversarial evaluation datasets, with 100% clean accuracy, 100% adversarial accuracy, and a 100% robustness score.

Overall, the system satisfies the main requirements of a GraphRAG pipeline. It uses graph-based retrieval to ground the answer generation process and provides a reliable, explainable, and reproducible question answering solution for the shared-housing domain.